Practical Exercise 2: AWS Storage Services

In this exercise, we will explore another foundational service in the Amazon Web Services (AWS) platform, namely, Amazon S3 (i.e. Amazon Simple Storage Service). The purpose of this exercise is for students to understand and utilise the AWS S3 - Cloud Object Storage service in their cloud applications. In this activity, you will be exploring the uses, capabilities, and benefits of using the AWS S3 or Simple Storage Service solution. You will also follow a practical guide to help you learn and practice how to store and access application data within the AWS S3 storage system.

To start building atop the AWS platform as part of these exercises, you need to ensure that you met the following requirements:

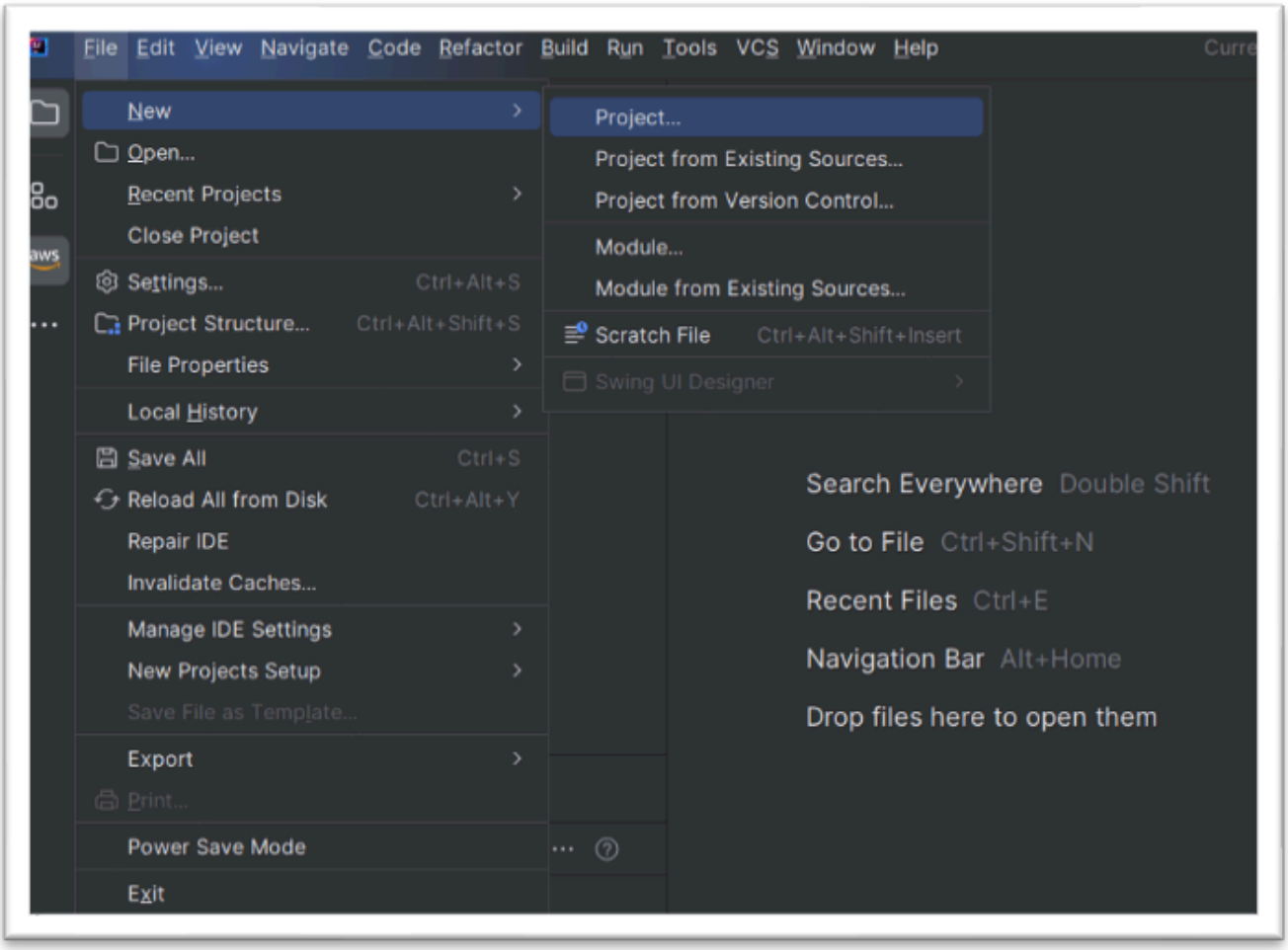
1. You signed up for AWS Academy platform.
2. You have Java installed in your local development environment.

Connecting the AWS Toolkit for IntelliJ IDEA to your AWS Account

Open IntelliJ IDEA – the Java IDE – that you installed in Week 2.

You can use either IntelliJ IDEA 2024.3.4.1 (Community Edition) or 2025.3.3 (Community Edition) , as both versions have been tested by the tutors, although slight differences may exist.

Step 1: From the menu bar at the top, select **File > New > Project** to create a new Java project. (**Note:** you will first need to tap the "≡" to reveal the menu bar at the top.)



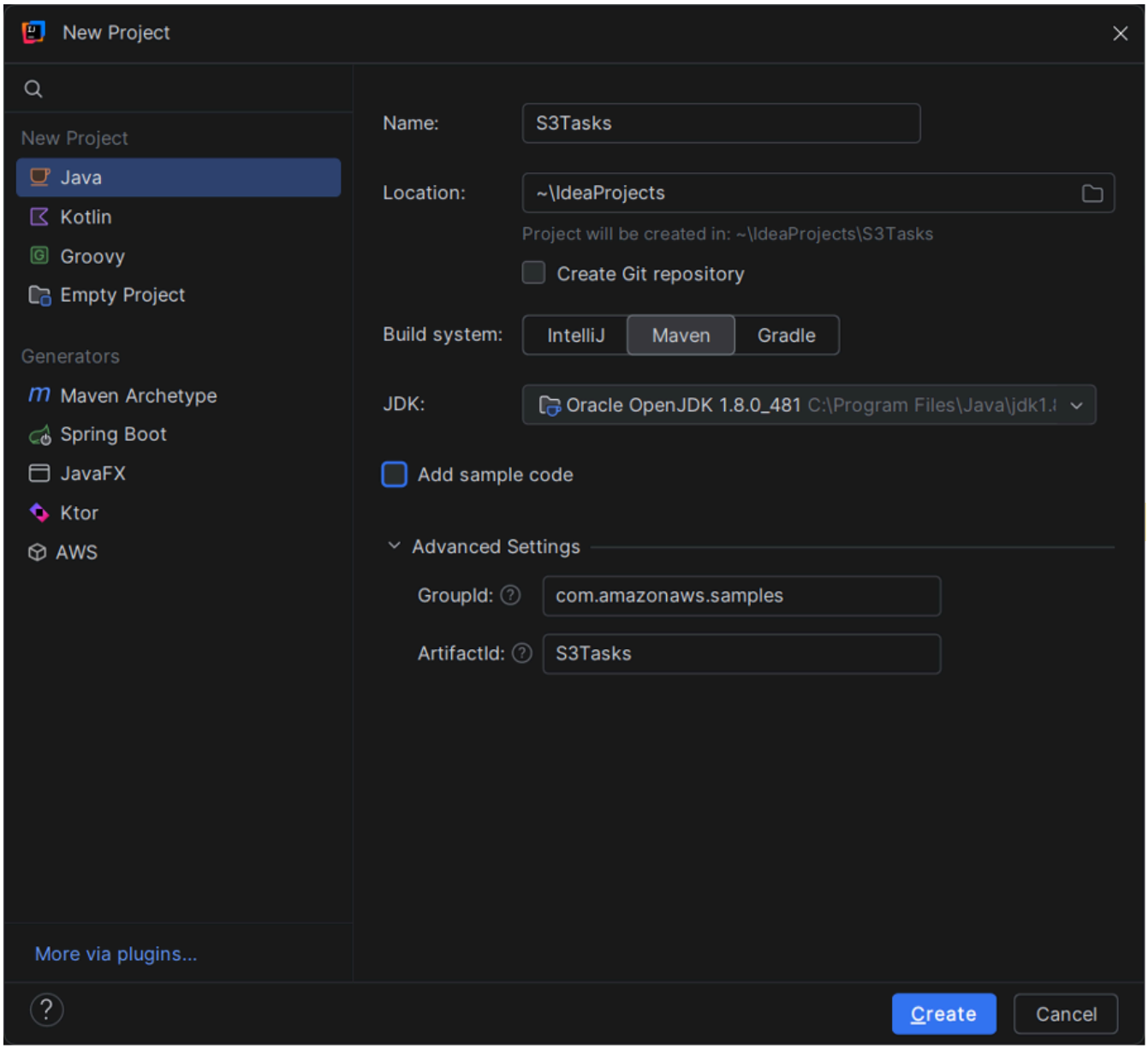
Step 2: In the Project name field, type a name for your project (e.g. **S3Tasks**).

Select **Language** as **Java**, **Build system** as **Maven**, **JDK** as **1.8**.

Next, untick the option *Add sample code*.

Now, expand the Advanced Settings section, and configure **GroupId** as **com.amazonaws.samples.**

Finally, tap the **Create** button at the bottom of the screen to create the project.



Note that the project is pre-configured and includes the AWS SDK libraries (e.g. .jar files).

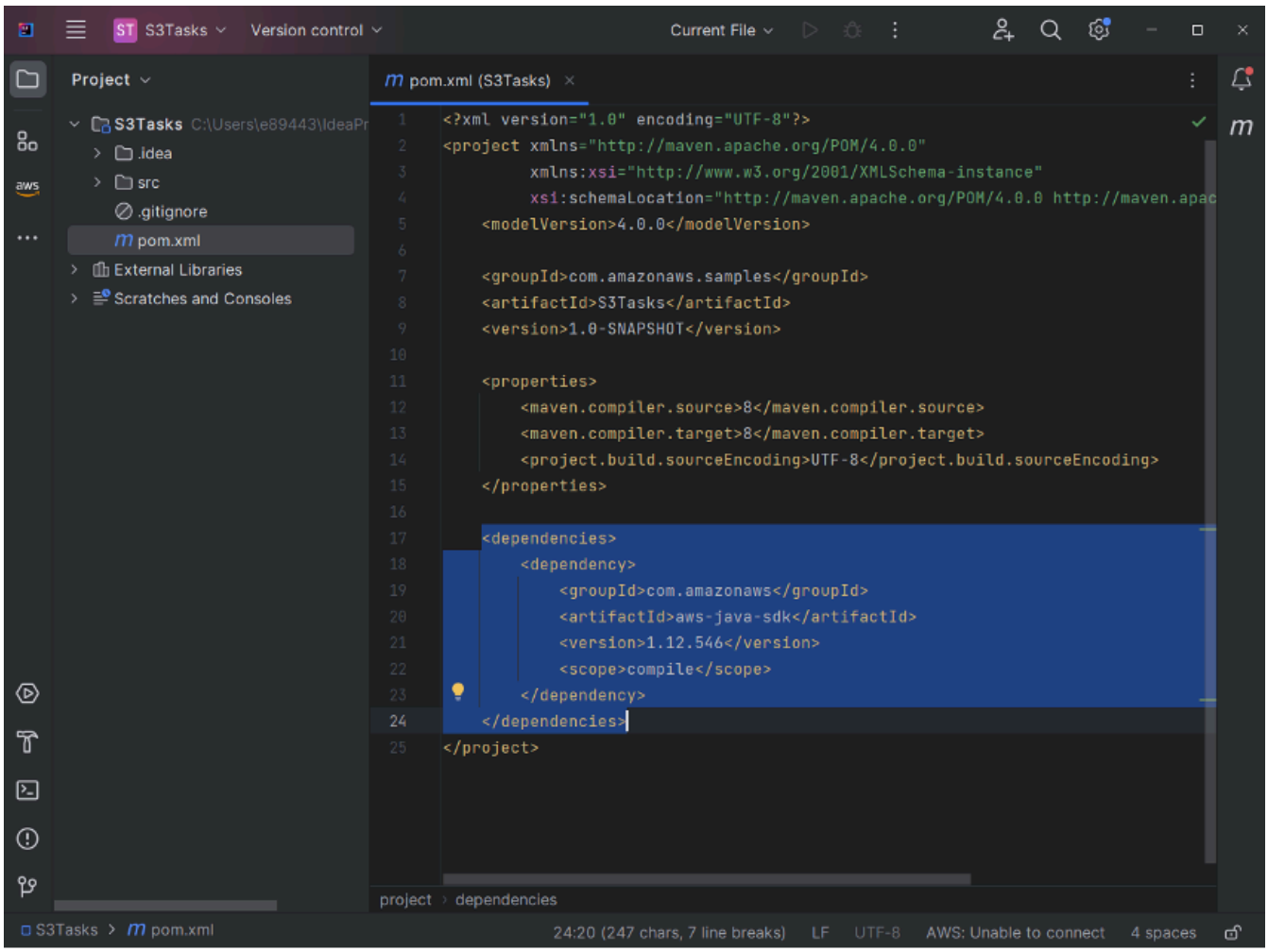
Step 3: Once the project has been created, open the **pom.xml** file and add the following configuration just below the **properties** element in the XML structure available.

This lets [maven](#) – our build tool – know that the AWS SDK of the specified version will be used in the Java project (i.e. **S3Tasks**) we just created.

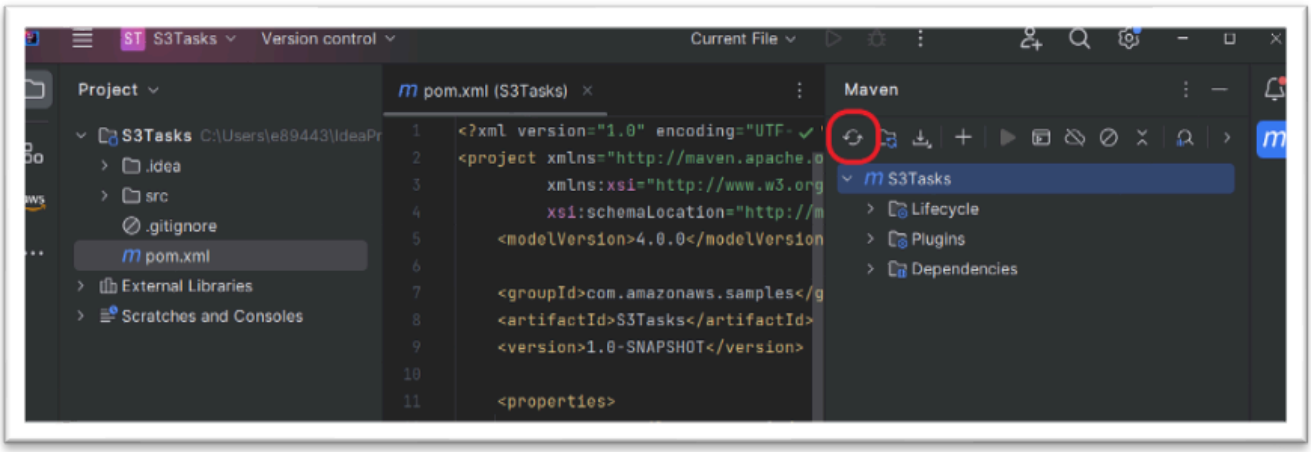
```
<dependencies>
    <dependency>
        <groupId>com.amazonaws</groupId>
        <artifactId>aws-java-sdk</artifactId>
        <version>1.12.546</version>
        <scope>compile</scope>
```

```
    </dependency>
</dependencies>
```

Your **pom.xml** should now look like what is shown in the image below.

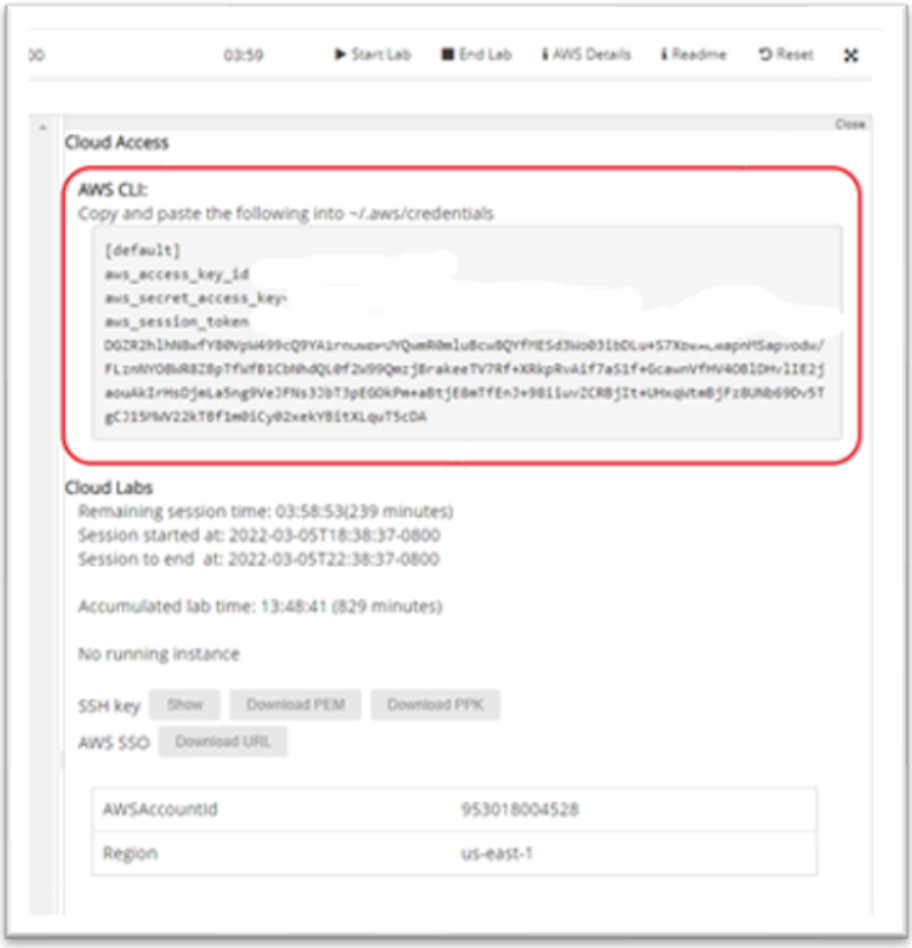


Step 4: Now, resolve the maven configuration by tapping the reload icon shown below. This downloads the AWS SDK related artifacts from a [maven repository](#) (i.e. a place where the AWS SDK related library is kept as a maven artifact.), and configures it in the Java project (i.e. **S3Tasks**) you just created.

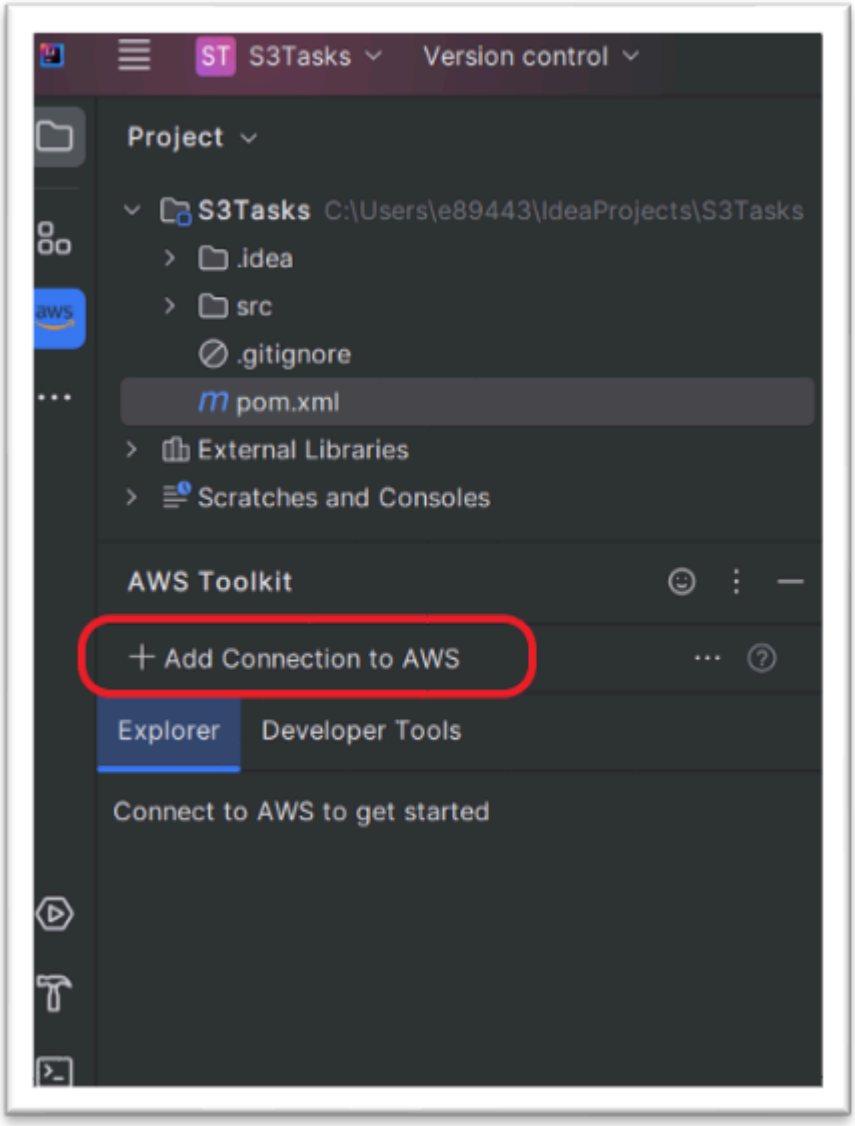


Step 5: Grab your **Access Key Id**, **Secret Key**, as well as **Session Access Key** associated with your current AWS Academy portal, as below. We

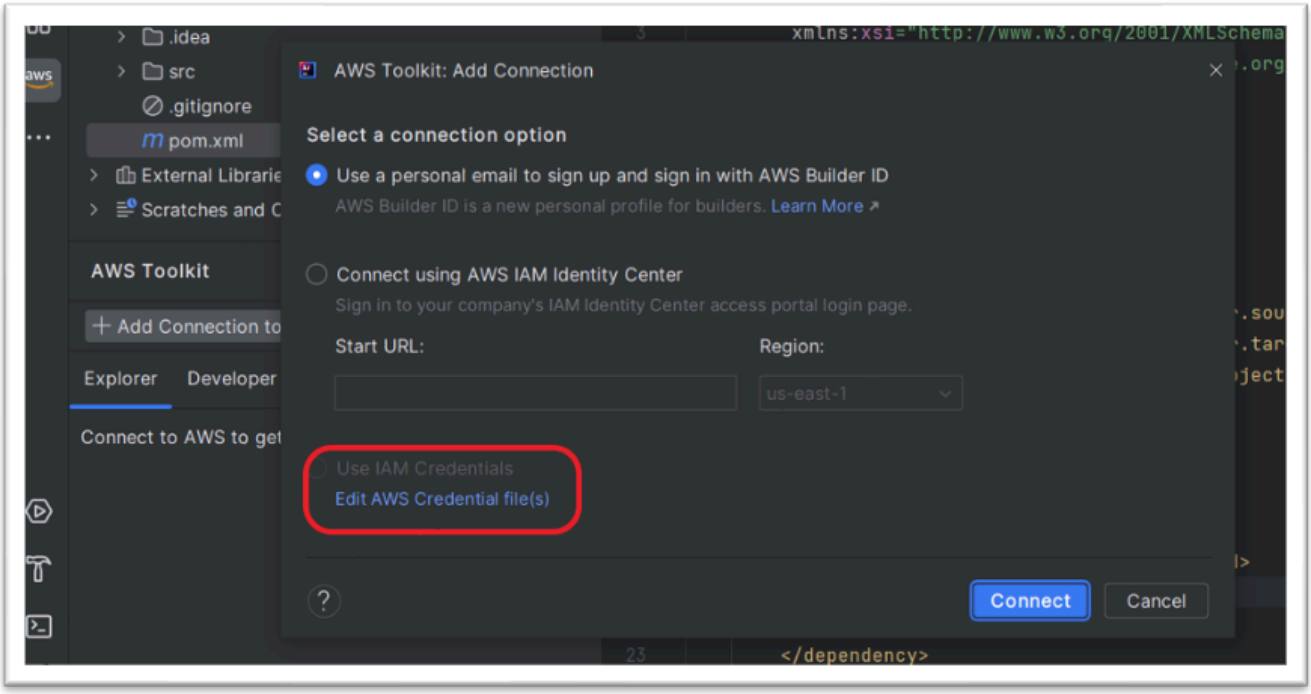
discussed about how these **short-lived credentials** allow us to access AWS services during the Week 2 practical exercise.

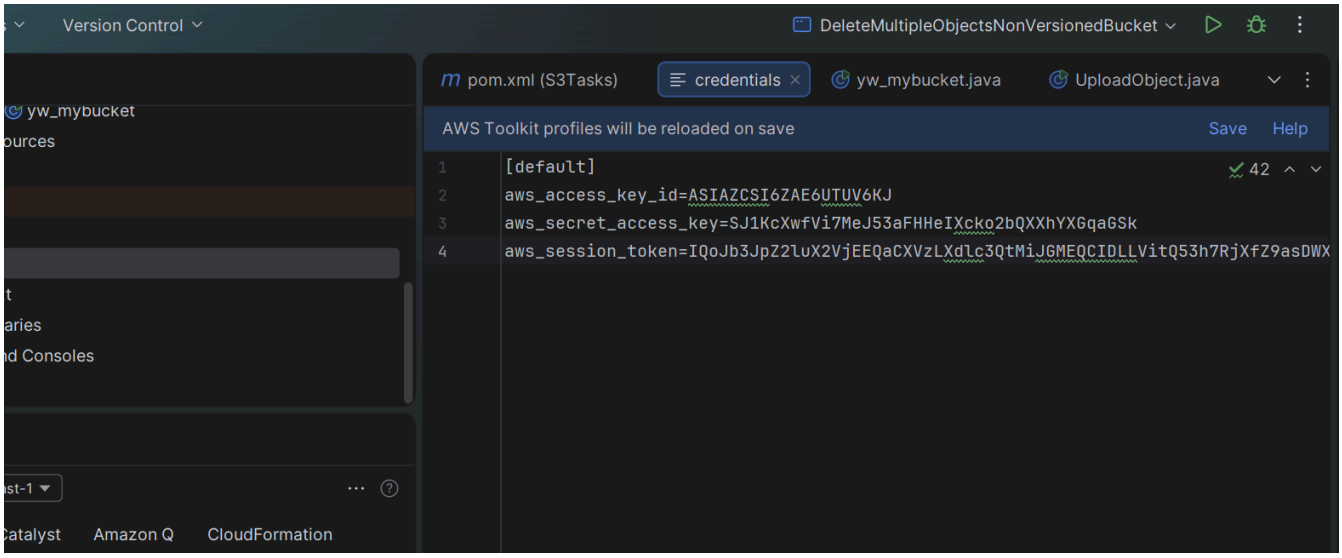


Step 6: Tap the **AWS Toolkit** icon from the left-hand side vertical menu to reveal the AWS Toolkit in your IDE. Now, select **Add Connection to AWS** to start setting up the credentials obtained in the previous step in your AWS Toolkit, as below.



Step 7: Next, tap the **Edit AWS Credential file(s)** link to configure the previously copied credentials to be used by your local development environment. Choose to create a credential file if the IDE prompts that it does not exist.

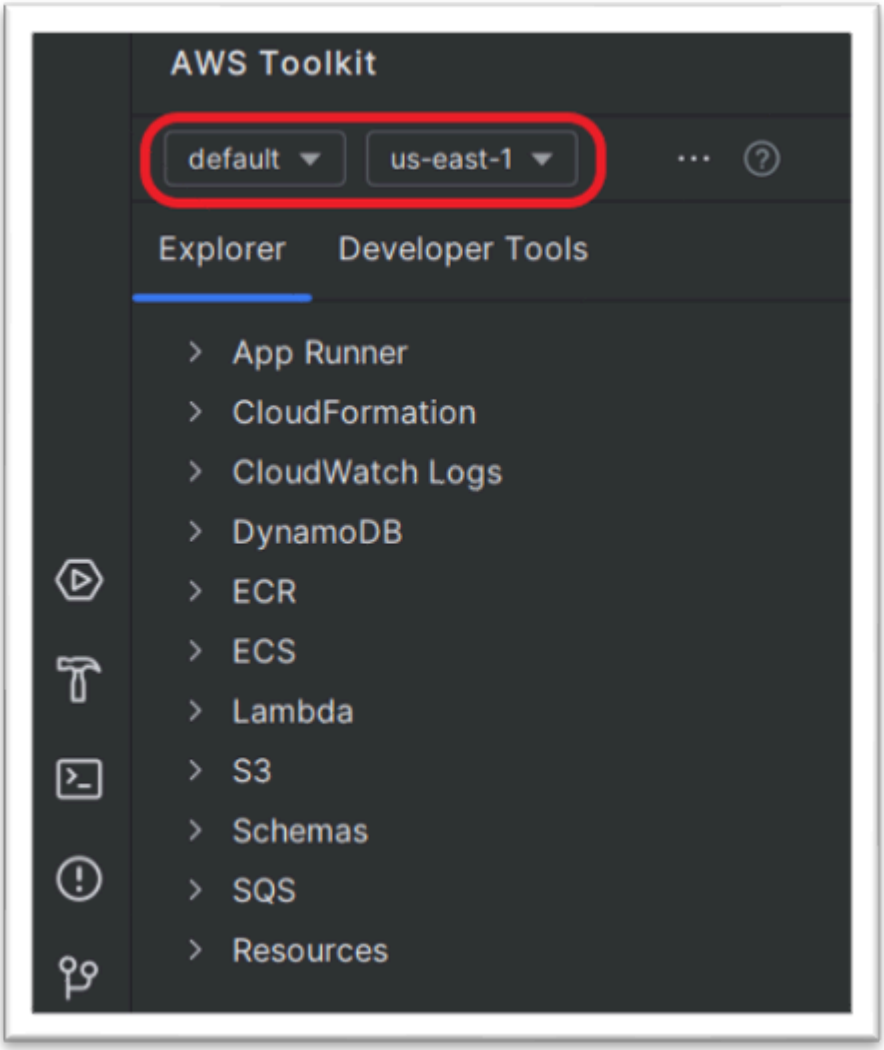




Step 8: Then tap **Save** to save the credentials you just configured in your file system. **Once done, these credentials can now be located in the directory** `%userprofile%\.aws` **(on Windows) and** `~/.aws/` **(on Linux and MacOSx).**

Step 9: If the previous step was successful, you should now be able to see the **default credentials profile** as well as the **default AWS region** automatically selected in your AWS Toolkit view.

In addition, you should also be able to observe that the **AWS Toolkit Explorer** now listing out various categories of AWS resources that you will later create as part of your learning tasks, which you can access through the AWS connection established with the steps above.



[Recommended] The credentials file is located at `~/.aws/credentials` on Linux or macOS, or at `C:\Users\USERNAME\.aws\credentials` on Windows. Later, when you want to update the credentials after an AWS Academy lab session expires and carry out *Steps 6 – 9* of this guides, the AWS Toolkit will automatically open `credentials` file.

Step 10: Now right-click on **S3Tasks > src > main > java** directory and create a new Java package named **com.amazonaws.samples**, if that is not available by default.

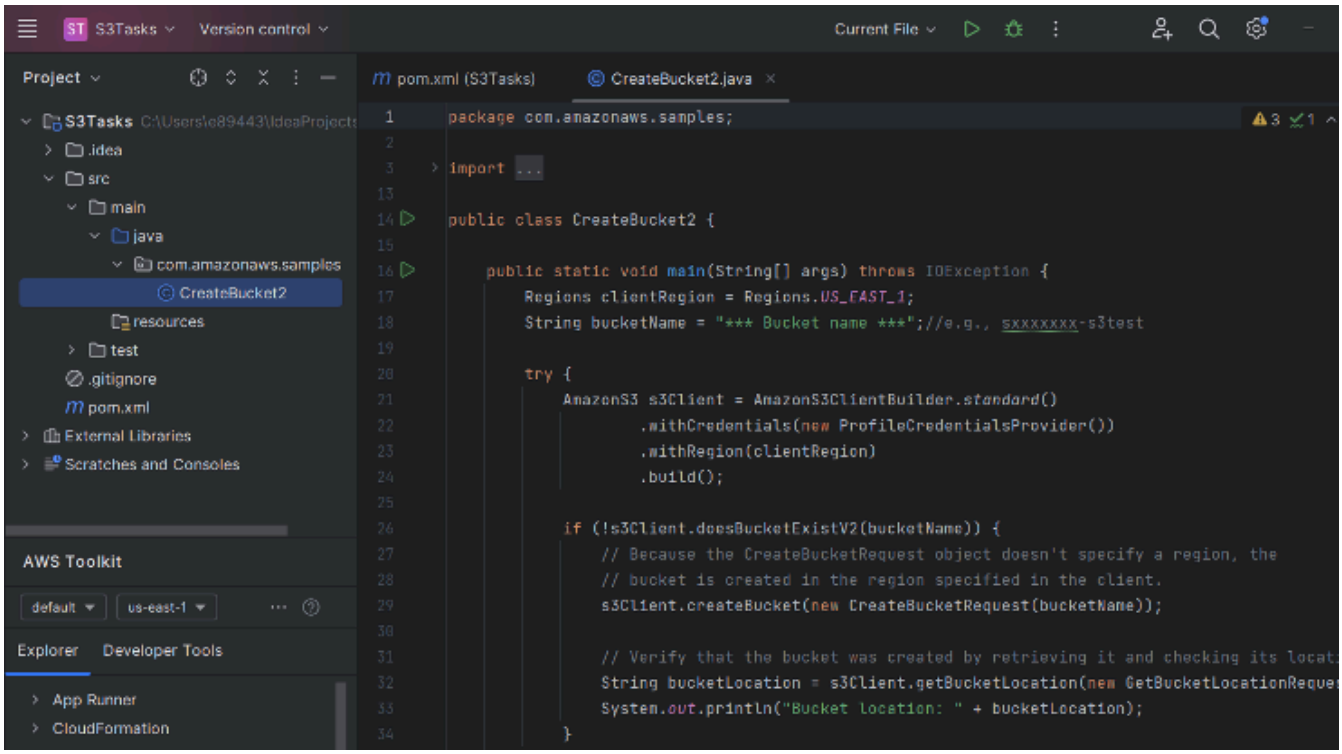
You are now ready to use your IDE configured above to perform various AWS S3 operations in the next section.

Activity 1: Create an AWS S3 Bucket

In this task, we will look to create an AWS S3 bucket, which is the *container* for storing objects in AWS S3.

Step 1: Open IntelliJ IDEA that you configured in the previous activity, which has the project **S3Tasks** loaded.

Step 2: Now, download the file **CreateBucket2.java** and copy paste it under the Java package **com.amazonaws.samples**. Then save it. Your project structure will now look like the following.



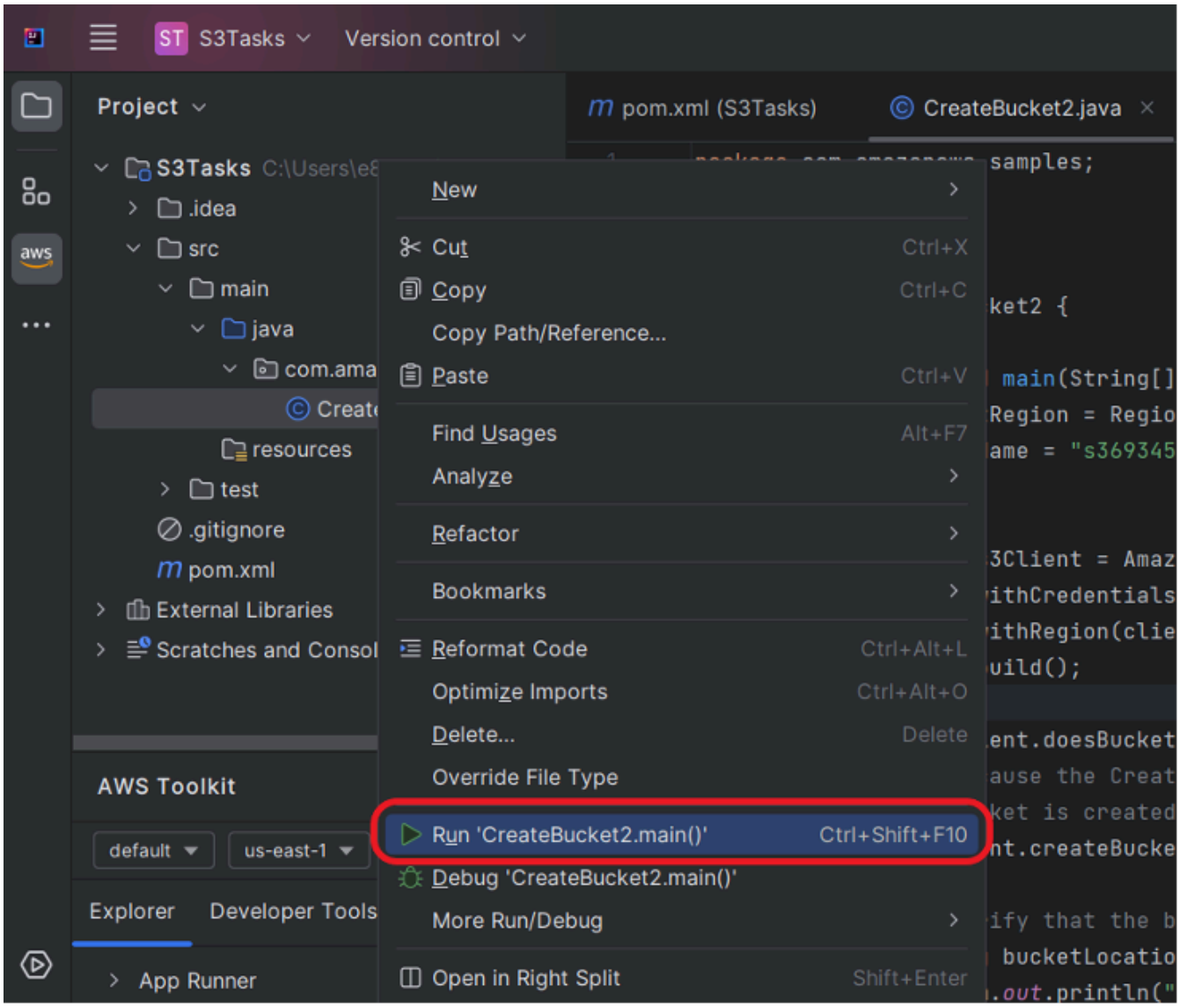
Step 3: Replace "*** *Bucket name* ***" in **CreateBucket2.java** with a suitable bucket name (must be unique, e.g. sxxxxxxxx-mybucket). Save your changes.

pom.xml (S3Tasks) CreateBucket2.java

```java
package com.amazonaws.samples;

import ...

public class CreateBucket2 {

    public static void main(String[] args) throws IOException {
        Regions clientRegion = Regions.US_EAST_1;
        String bucketName = "sxxxxxxxx-mybucket";//e.g., sxxxxxxx-s3test

        try {
            AmazonS3 s3Client = AmazonS3ClientBuilder.standard()
                    .withCredentials(new ProfileCredentialsProvider())
                    .withRegion(clientRegion)
                    .build();

```

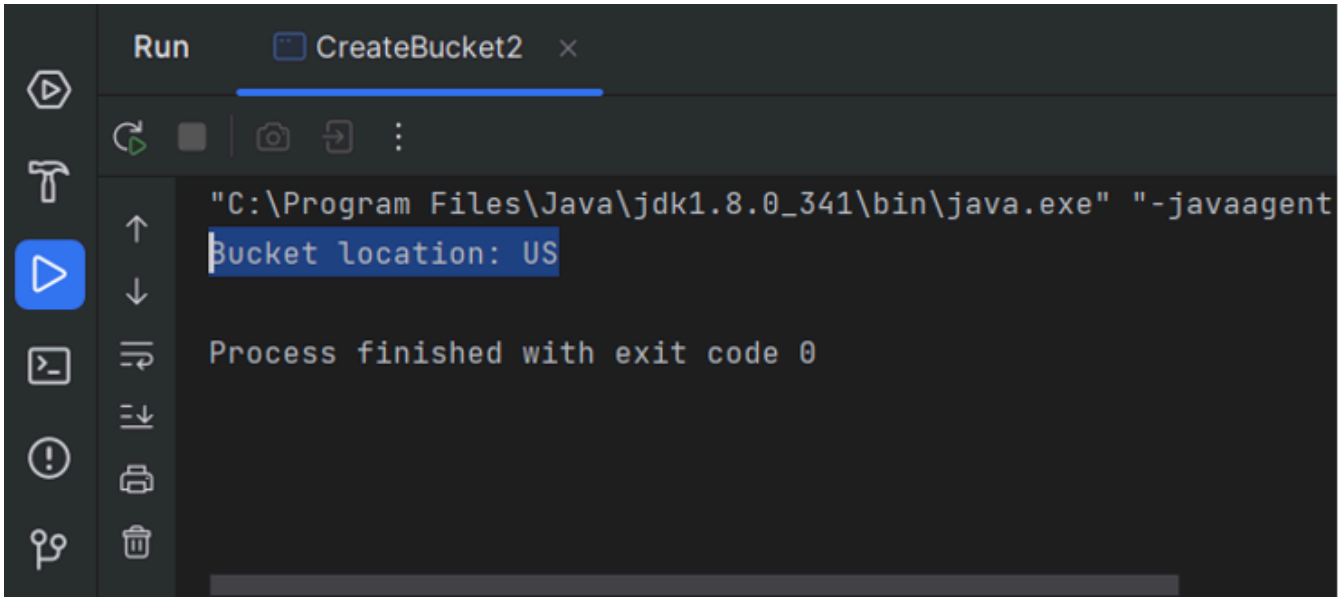
Step 4: Make sure your region is set to **Regions.US_EAST_1** or **Regions.US_WEST_2** in your **CreateBucket2.java** as AWS Academy only allows you to create resources within the two AWS regions.

Step 5: Right click **CreateBucket2** on the Project Explorer. Then, select '**RunCreateBucket.main()**', as below.



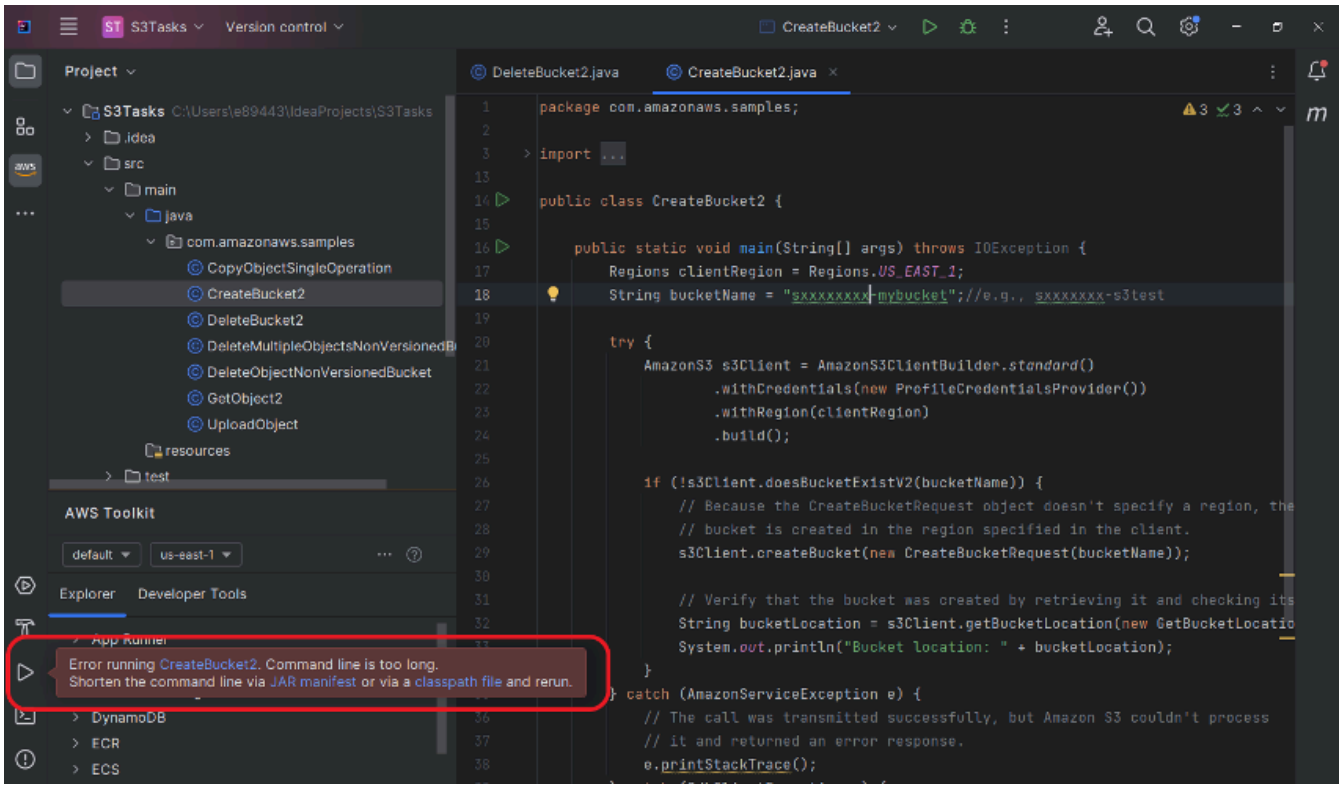
See the **Troubleshooting Tips** section below if you run into errors, while attempting this step.

Step 6: The application will start to run. Wait until you see a success message in IntelliJ IDEA console, as below.



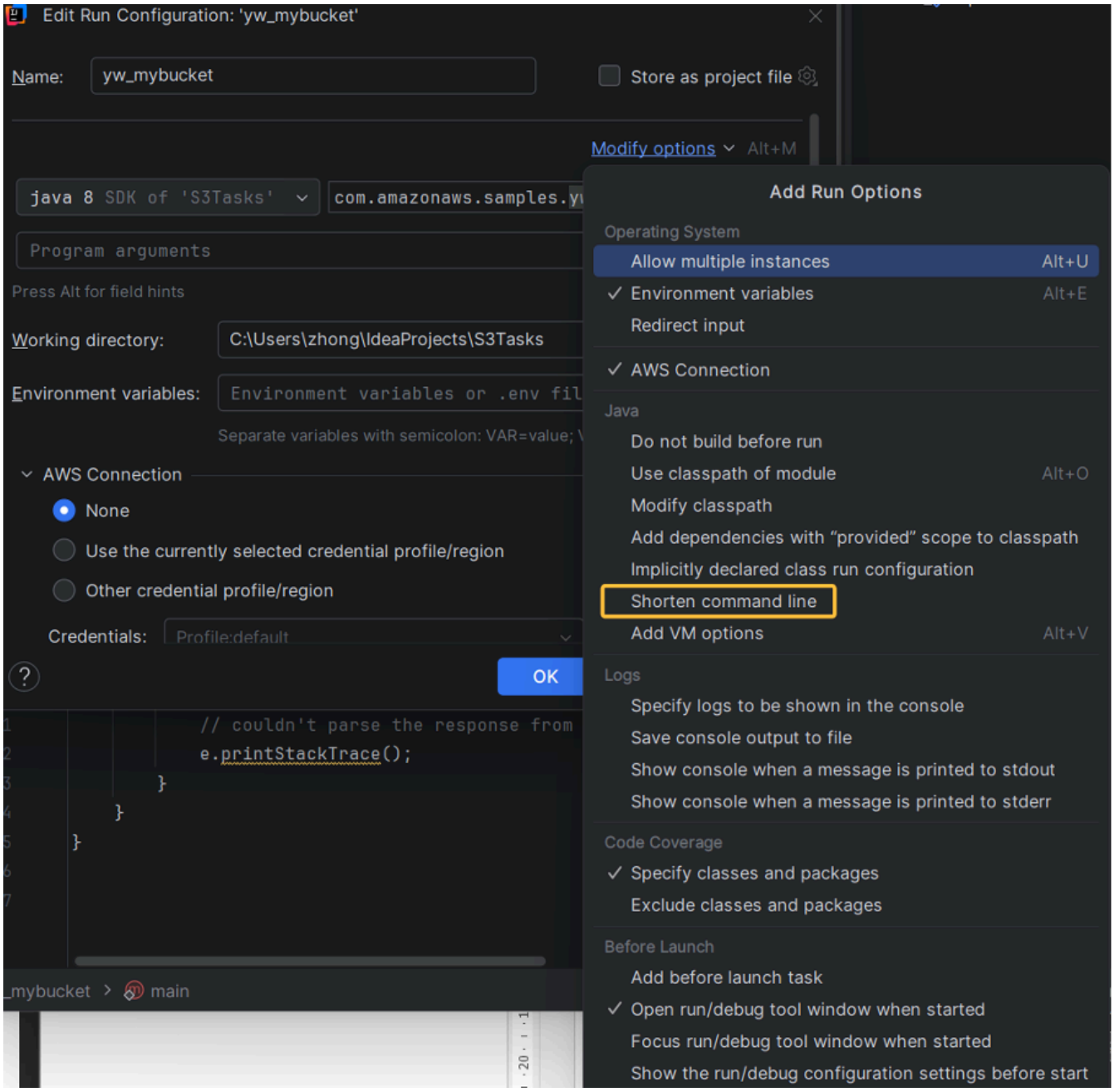
Troubleshooting Tips

See if you run into an error that looks like the following.

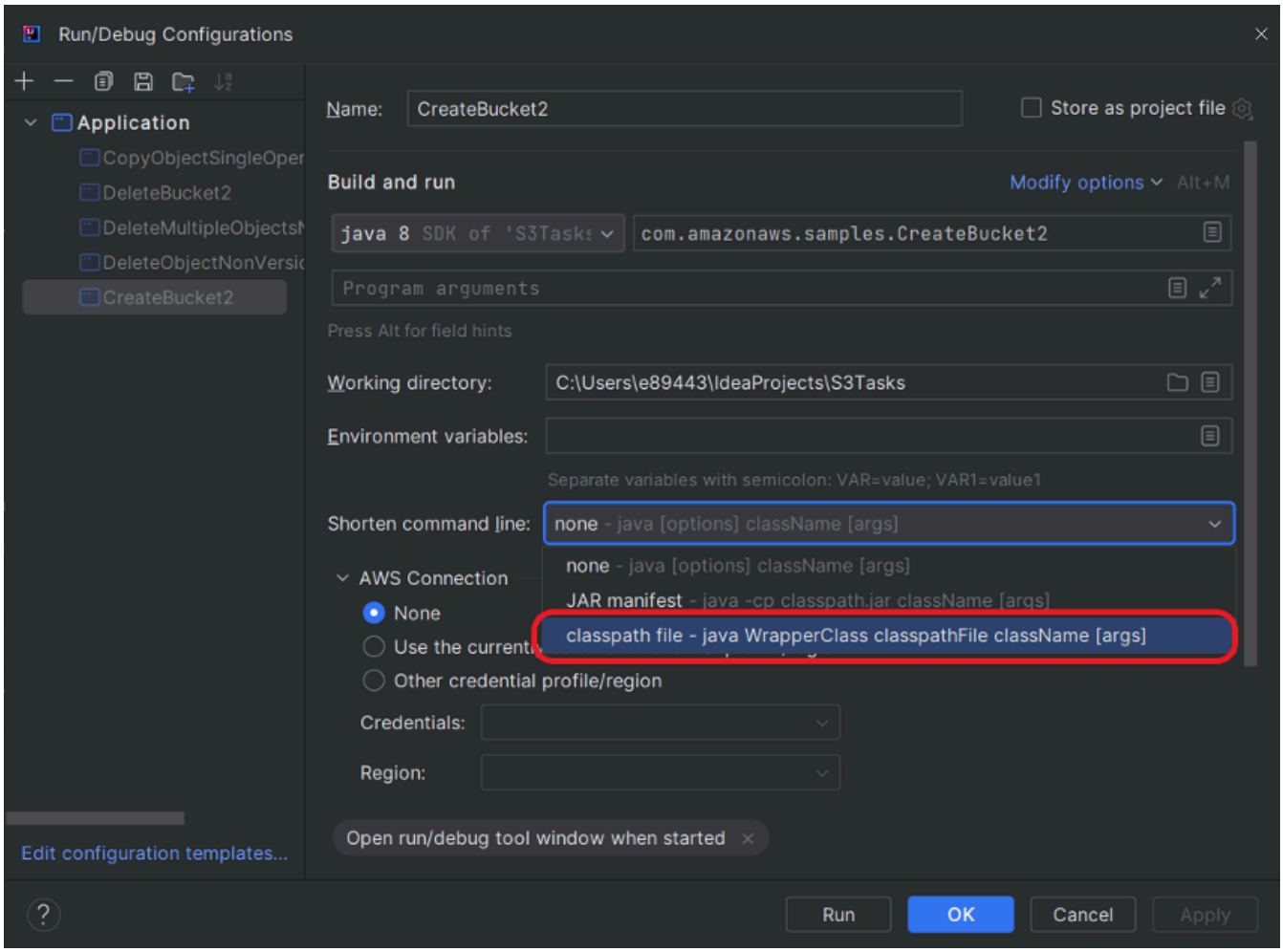


This error usually occurs when a command (i.e. running CreateBucket2.java) you're trying to execute has exceeded the maximum character limit for command-line input. This can happen when you have a very long command or when you're working with a large number of files as arguments to a command. On this occasion, IntelliJ IDEA appending multiple VM arguments while running your Java applications (e.g. CreateBucket2.java) is the culprit.

To get around the issue, right click on **CreateBucket2** file in the **Project Explorer**.



Then, from the **Run Configuration** view, select **classpath file** as the value of the **Shorten command** *line* property.



Tap **OK** and re-run the application as described in *Step 5.*

Activity 2: Upload Data into an AWS S3 Bucket

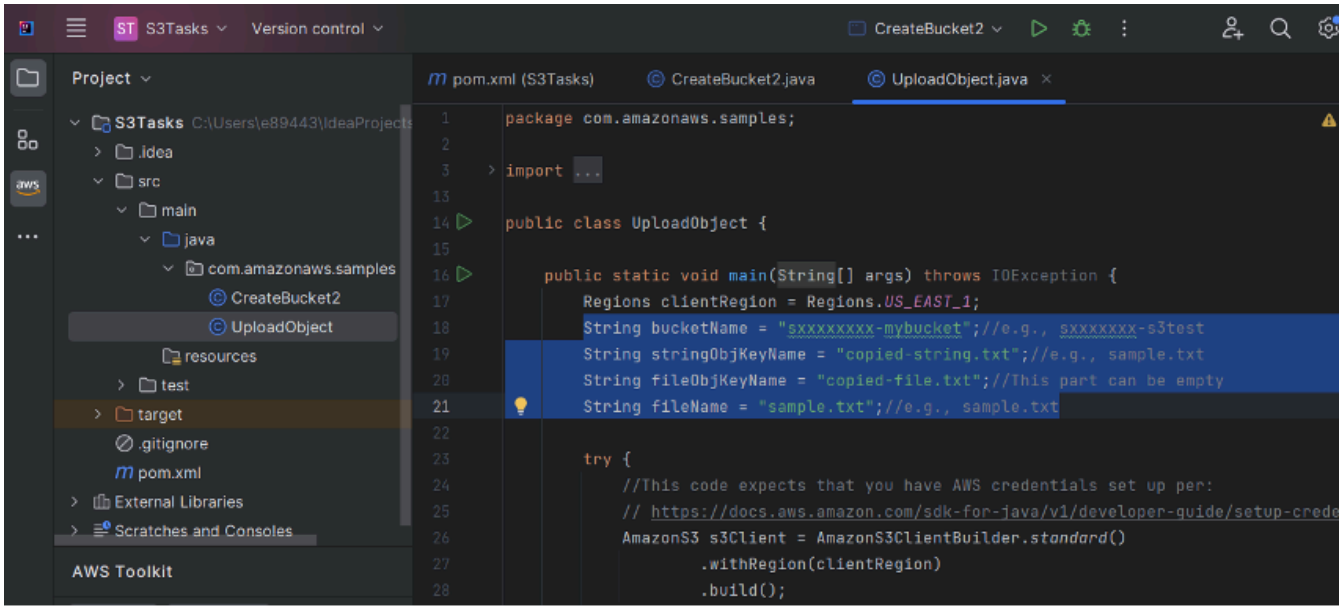
In this task, we will look at how to upload different types of **objects** (e.g. text, files) into an AWS S3 bucket.

The following task creates two objects. The first object has a text string as data, and the second object is a file. We create the first object by specifying the bucket name, object key, and text data directly in a call to `AmazonS3Client.putObject()`. We then create the second object by using a `PutObjectRequest` that specifies the bucket name, object key, and file path. The `PutObjectRequest` also specifies the `ContentType` header and title metadata.

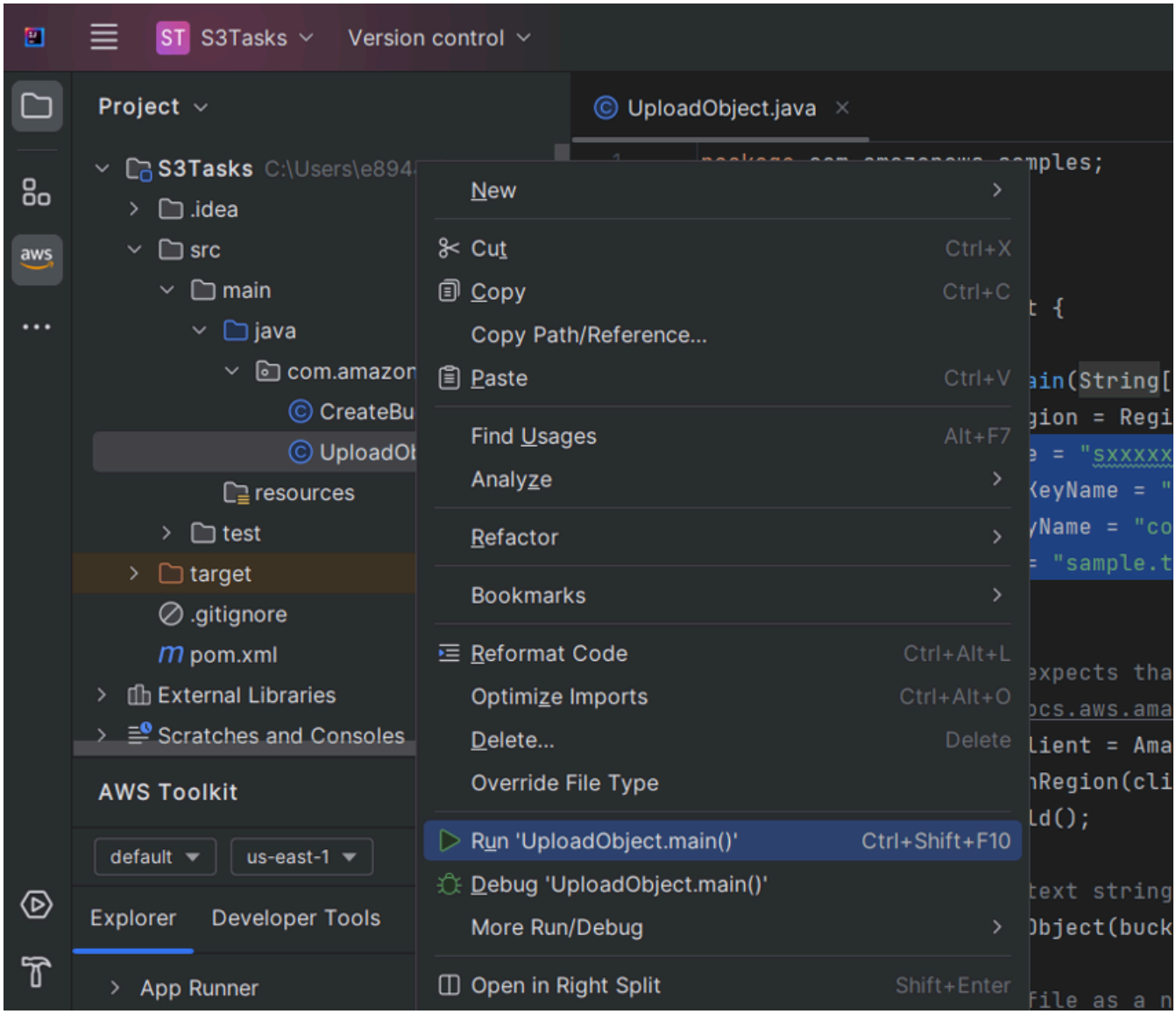
Step 1: Download the file **UploadObject.java** and copy-paste it under the Java package **com.amazonaws.samples** of your project – **S3Tasks**.

Step 2: Download the file **sample.txt** and copy-past it into the **S3Tasks** project, this time, however, into the **root directory** (e.g. S3Tasks/) of the project.

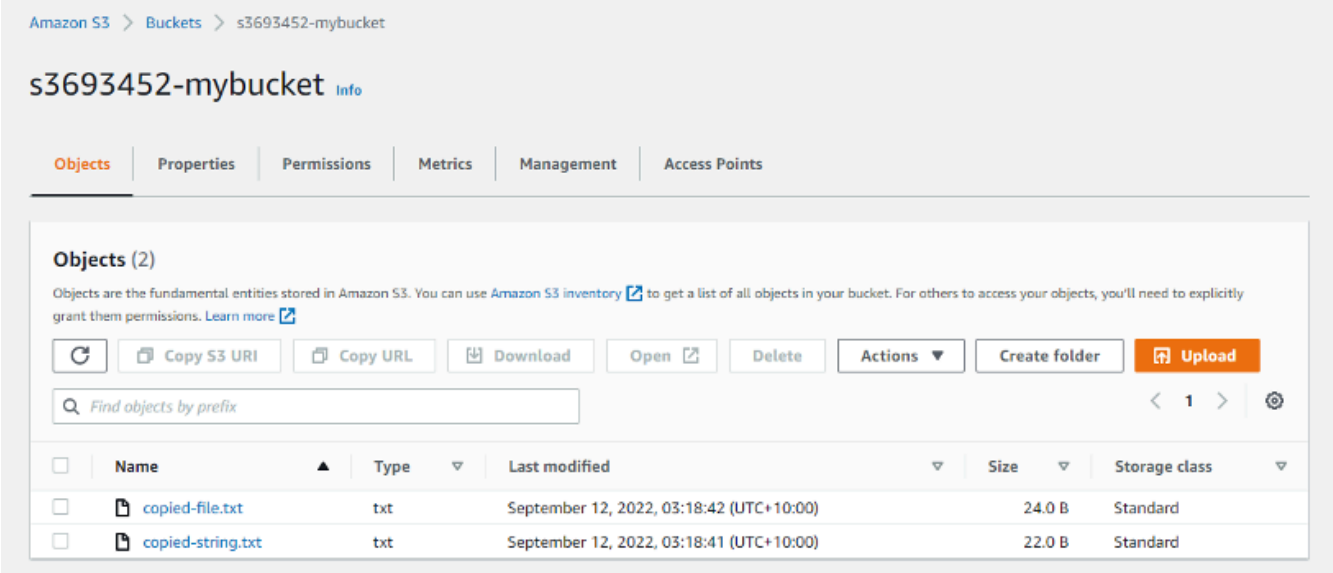
Step 3: Replace "*** *Bucket name* ***" with the name of the bucket you created previously (e.g. sXXXXXXX-mybucket). Also replace "*** String object key name ***" with "**copied-string.txt**". The aforementioned parameter defines how your string object would look like in your AWS S3 bucket once it has been uploaded. Meanwhile, you further need to replace "*** File object key name ***" and "*** Path to file to upload ***" with "**copied-file.txt**" and "**sample.txt**", respectively. These two parameters define how your uploaded file would look like in your AWS S3 bucket and where to find it in the local file system, respectively.



Step 4: Right click **UploadObject** on the Project Explorer. Then, select '**Run UploadObject.main()**', as below.



Step 5: The application will start to run. Wait until it finishes and check the AWS S3 Console. You will see a view like the following.



Troubleshooting Tips

See if you run into an error that looks like the following.

```
Exception in thread "main" java.lang.NoClassDefFoundError: javax/xm
l/bind/JAXBException
    at com.amazonaws.services.s3.AmazonS3Client.putObject(AmazonS3C
lient.java:1759)
    at com.amazonaws.services.s3.AmazonS3Client.putObject(AmazonS3C
lient.java:3416)
    at com.amazonaws.samples.UploadObject.main(UploadObject.java:3
1)Caused by: java.lang.ClassNotFoundException: javax.xml.bind.JAXBE
xception
    at java.base/jdk.internal.loader.BuiltinClassLoader.loadClass(B
uiltinClassLoader.java:641)
    at java.base/jdk.internal.loader.ClassLoaders$AppClassLoader.lo
adClass(ClassLoaders.java:188)
    at java.base/java.lang.ClassLoader.loadClass(ClassLoader.java:5
20)
    ... 3 more
```

This usually indicates a mismatch between the expected and actual versions of the Java Runtime Environment. The UploadObject.java expects the version of Java you are running to be 1.8. If you are running a more recent version, you may need to add the following configuration into the file **pom.xml** of your project between the `<dependencies></dependencies>` tags, as below.

```
<!-- API, java.xml.bind module -->
    <dependency>
        <groupId>jakarta.xml.bind</groupId>
        <artifactId>jakarta.xml.bind-api</artifactId>

        <version>2.3.2</version>
    </dependency>
    <!-- Runtime, com.sun.xml.bind module -->
    <dependency>
        <groupId>org.glassfish.jaxb</groupId>
        <artifactId>jaxb-runtime</artifactId>
        <version>2.3.2</version>
    </dependency>
```

Activity 3: Downloading Data from an AWS S3 Bucket

In this task, we will look at how to download objects uploaded into an AWS S3 bucket.

When you download an object through the AWS SDK for Java, Amazon S3 returns all the object's metadata and an input stream from which to read the object's contents. To retrieve an object, you do the following:

1. Run the `AmazonS3Client.getObject()` method, providing the bucket name and object key in the request.
2. Run one of the `S3Object` instance methods to process the input stream.

Note: Your network connection needs to remain open until you read all the data or close the input stream. We recommend that you read the content of the stream as quickly as possible.

These are some of the variations you might use.

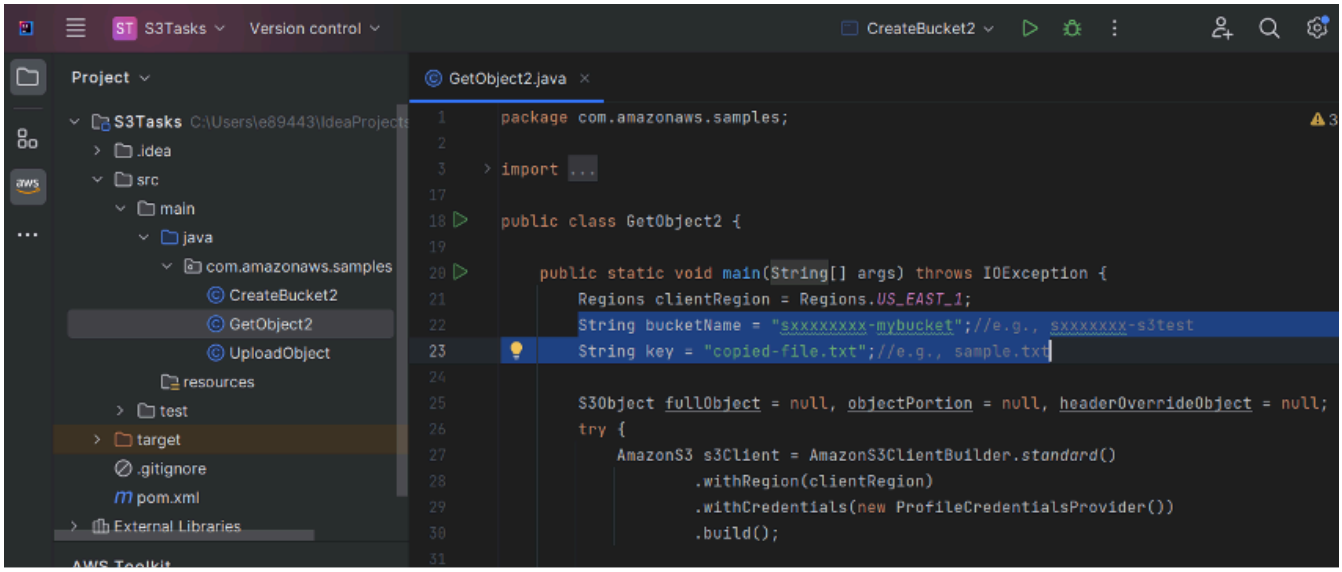
1. Instead of reading the entire object, you can read only a portion of the object data by specifying the byte range that you want in the request.
2. You can optionally override the response header values (see **Downloading objects**) by using a `ResponseHeaderOverridesobject` and setting the corresponding request property. For example, you can use this feature to indicate that the object should be downloaded into a file with a different file name than the object key name.

The following task retrieves an object from an AWS S3 bucket in three ways: first, as a complete object, then as a range of bytes from the object, then as a complete object with overridden response header values. For more information about getting objects from AWS S3, see **GetObject**.

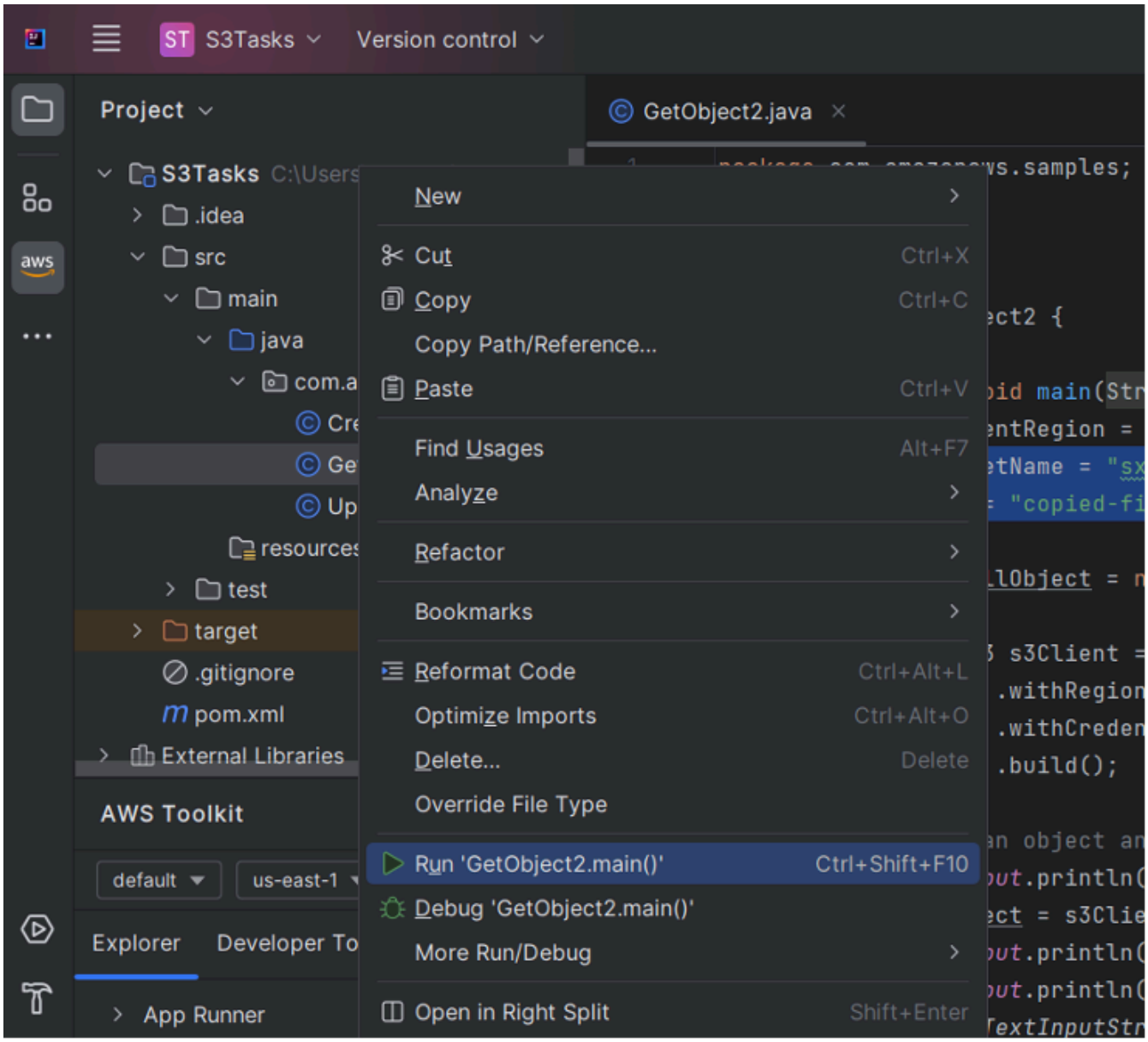
To run this task, follow the steps below.

Step 1: Download the file **GetObject2.java** and copy-paste it under the Java package *com.amazonaws.samples* of your project – **S3Tasks**.

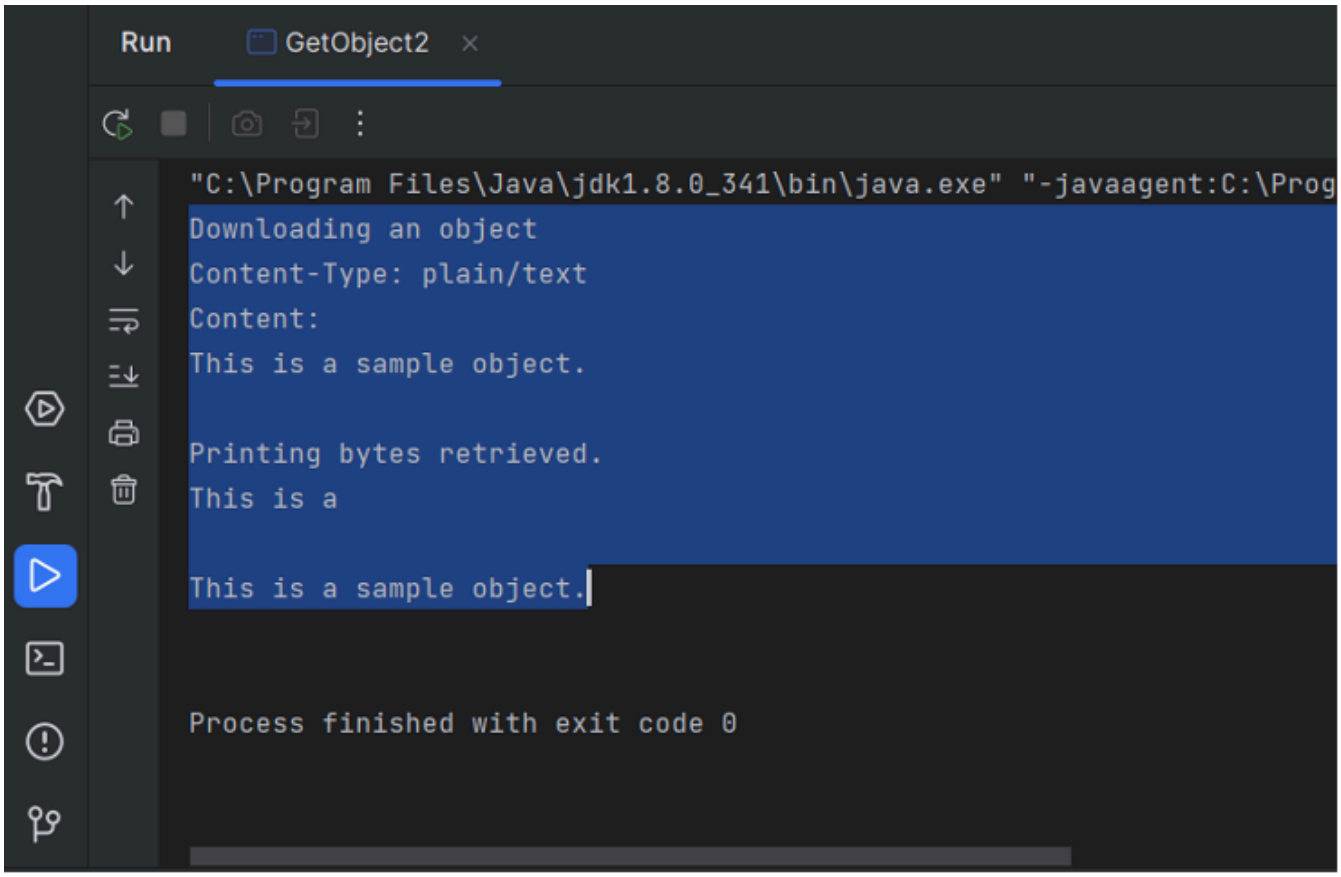
Step 2: Replace "*** *Bucket name* ***" with the name of the bucket you created previously (e.g. sxxxxxxxx-mybucket). Also replace "*** Object key ***" with "**copied-file.txt**". The object key should match that of an object that is already uploaded to your bucket. **Save your changes.**



Step 3: Right click **GetObject2** on the Project Explorer and then select '**Run GetObject2.main()**', as below.



Step 4: The application will start to run. Wait until it finishes. You will see an output like the following.


Run
GetObject2
"C:\Program Files\Java\jdk1.8.0_341\bin\java.exe" "-javaagent:C:\Prog
Downloading an object
Content-Type: plain/text
Content:
This is a sample object.
Printing bytes retrieved.
This is a
This is a sample object.
Process finished with exit code 0

Activity 4: Other useful AWS S3 Data Access Operations

Continuing from what we covered as part of the previous two sections, in this task, we will look at how to perform a range of other useful AWS S3 data access operations.

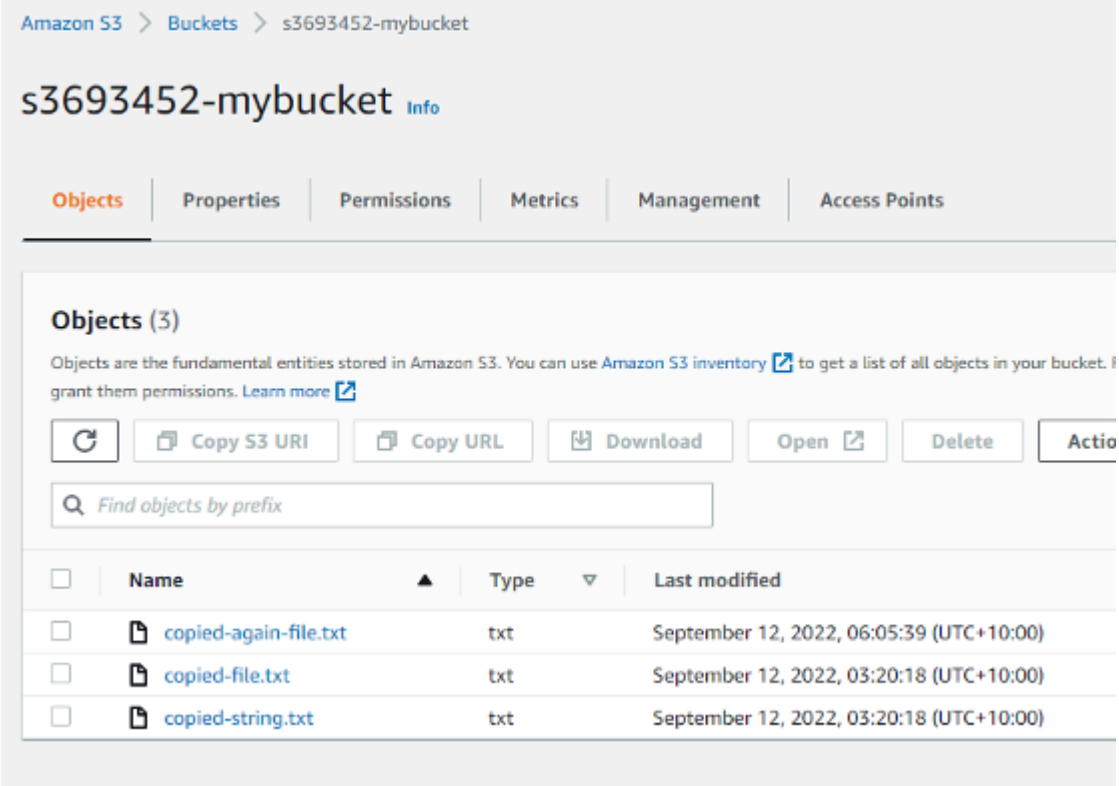
We will now look at the following useful AWS S3 data access operations:

1. Copy an object into a bucket.
2. Delete a single object from a bucket.
3. Delete multiple objects from a bucket.
4. Delete a bucket.

4.1 Copy an Object into a Bucket

The following example shows how to copy an object in Amazon S3 using the AWS SDK for Java.

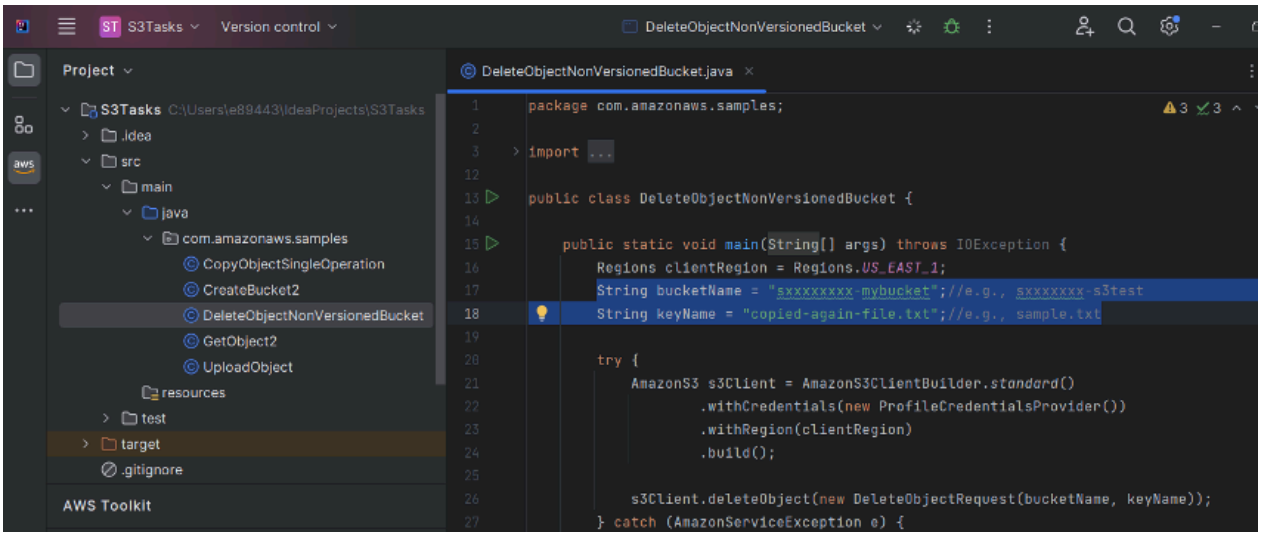
1. Download the file **java** and copy-paste it under the Java package **com.amazonaws.samples**of your project – **S3Tasks**.
2. Replace "*** *Bucket name* ***" with the name of the bucket you created previously (e.g. sXXXXXXX-mybucket). Also replace "*** Source object key ***" with "**copied-file.txt**". The source object key should match that of an object that is already uploaded to your bucket. Save your changes. Next, replace "*** Destination object key ***" as "**copied-again-file.txt**".
3. Right click on **CopyObjectSingleOperation.java** file. Then Select **Run as>Java application**. The application will start to run. Wait until it finishes. You will see a view like the following in AWS S3 Console.



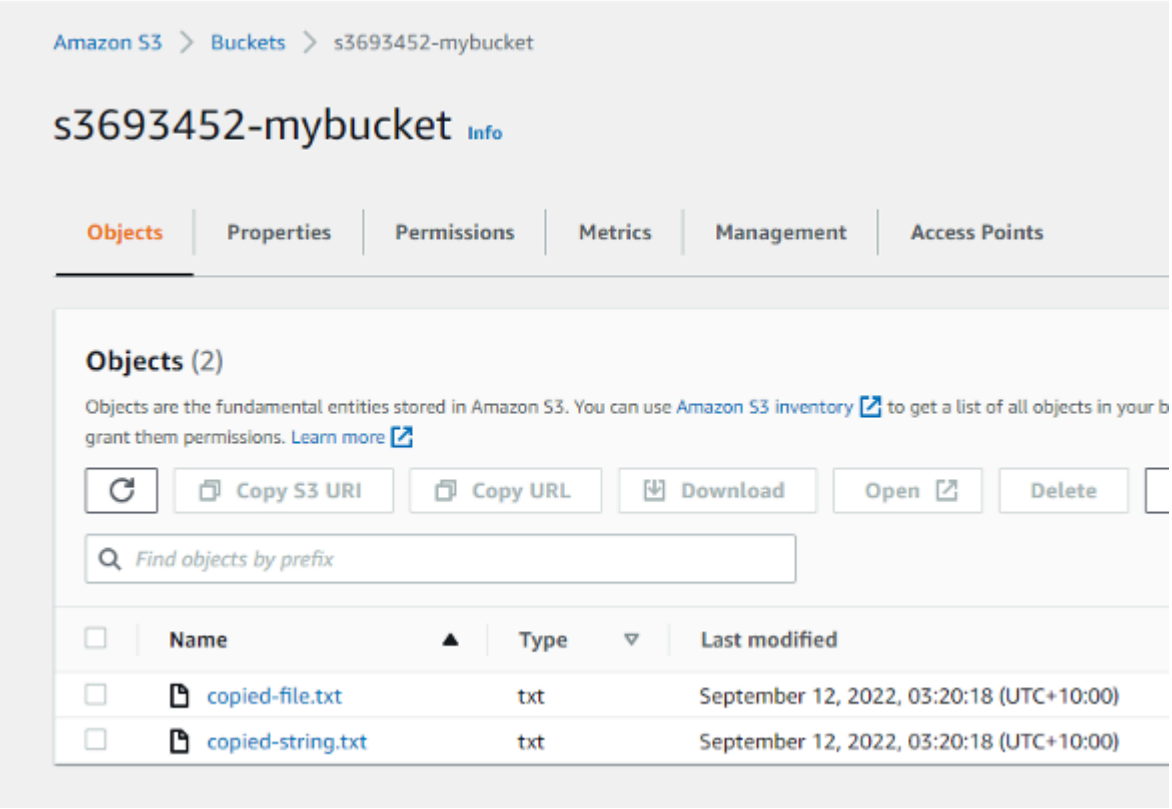
4.2 Delete an Object from a Bucket

The following task deletes an object from a bucket. This task assumes that the bucket is not versioning-enabled, and the object doesn't have any version IDs. In the delete request, you specify only the object key and not a version ID.

1. Download the file **DeleteObjectNonVersionedBucket.java** and copy-paste it under the Java package **com.amazonaws.samples** of your project – **S3Tasks**.
2. Replace "*** *Bucket name* ***" with the name of the bucket you created previously (e.g. sXXXXXXX-mybucket). Also replace "*** Key name ****" with "**copied-again-file.txt**". The key name should match that of an object that is already available in your bucket.



3. Right click on **DeleteObjectNonVersionedBucket.java** file. Then Select **Run as>Java application**. The application will start to run. Wait until it finishes. You will see a view like the following in AWS S3 Console. Note how the object **copied-again-file.txt** generated in the previous task disappeared.



4.3 Delete Multiple Objects from a Bucket

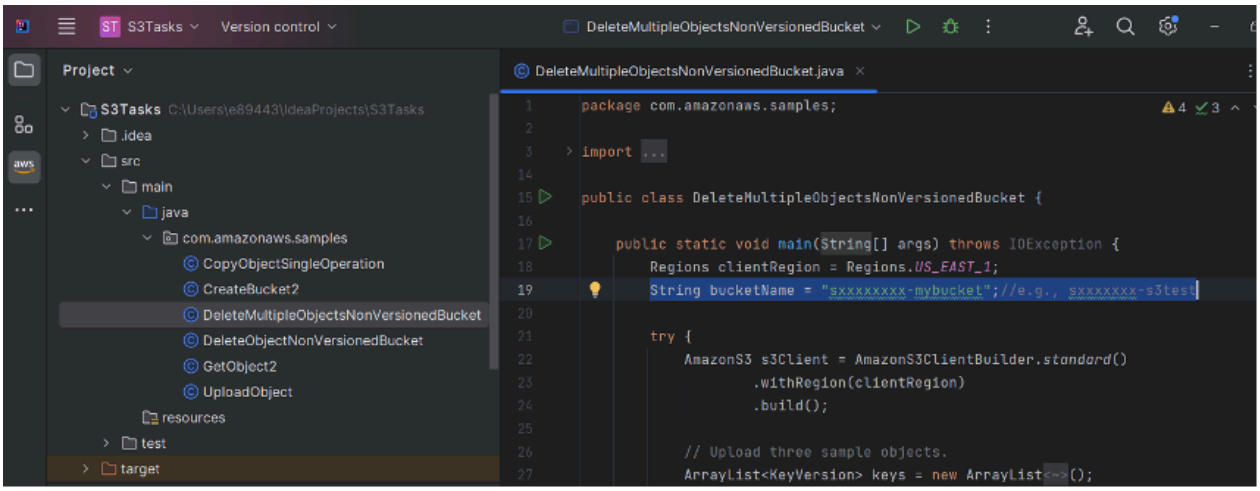
The AWS SDK for Java provides the `AmazonS3Client.deleteObjects()` method for deleting multiple objects. For each object that you want to delete, you specify its key name.

The following task uses the Multi-Object Delete API to delete objects from a bucket that is not version-enabled. The task uploads sample objects to the

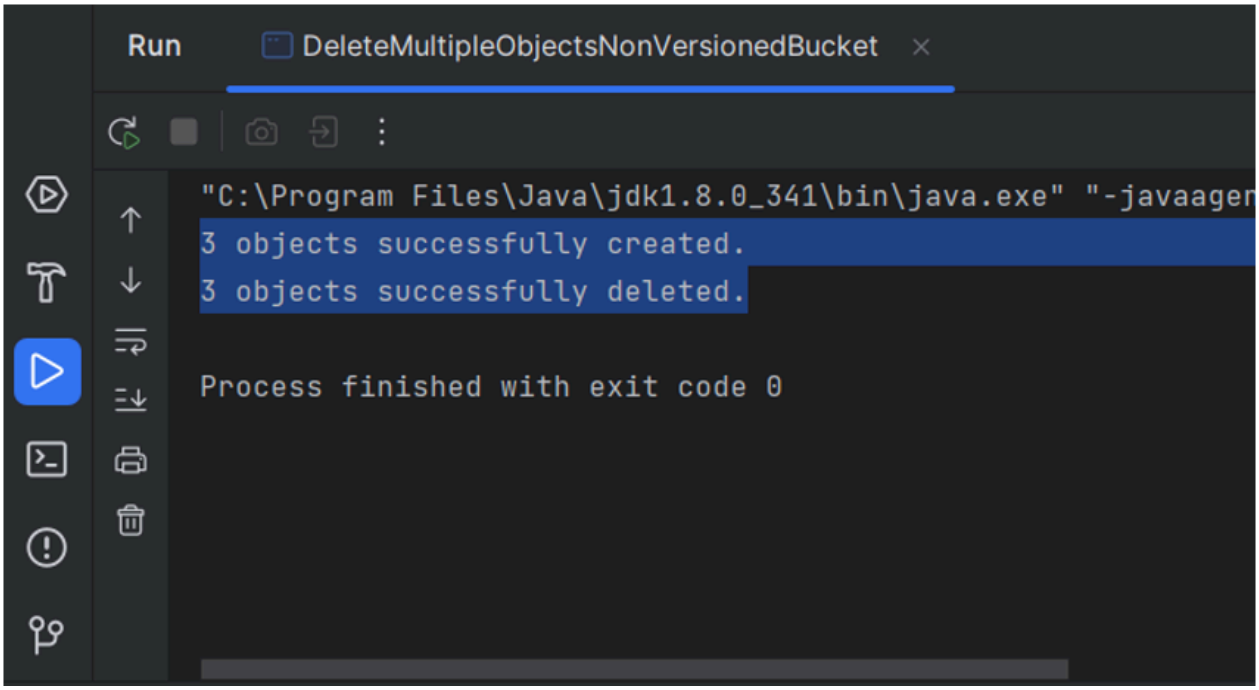
bucket and then uses the `AmazonS3Client.deleteObjects()` method to delete the objects in a single request. In the `DeleteObjectsRequest`, the example specifies only the object key names because the objects do not have version IDs.

For more information about deleting objects, see **Deleting multiple objects**.

1. Download the file **DeleteMultipleObjectsNonVersionedBucket.java** and copy-paste it under the Java package **com.amazonaws.samples**of your project – **S3Tasks**.
2. Replace "*** *Bucket name* ***" with the name of the bucket you created previously (e.g. sXXXXXXX-mybucket).



3. Right click on **DeleteMultipleObjectsNonVersionedBucket.java** file and select **Run as>Java application**. The application will start to run. Wait until it finishes. You will see the following output.

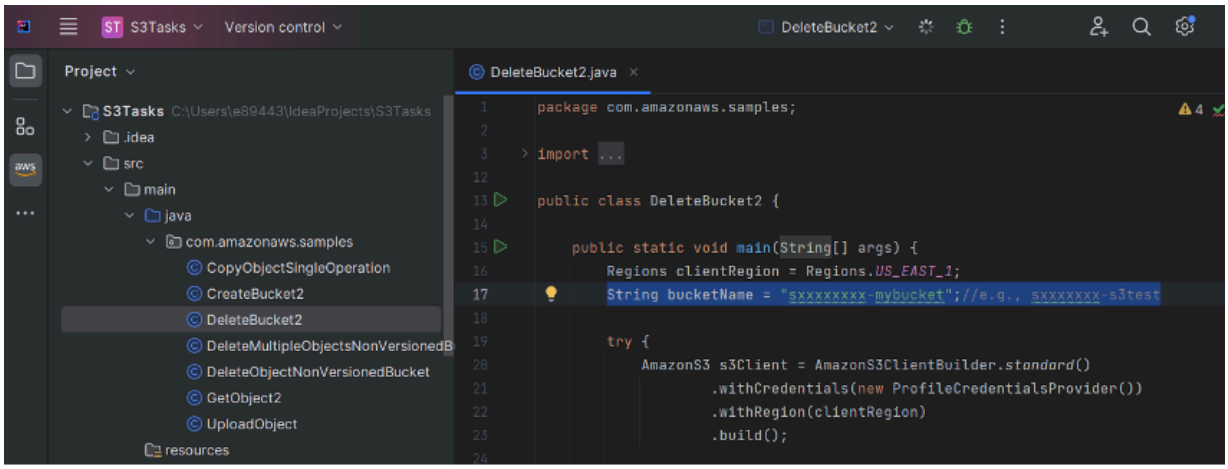


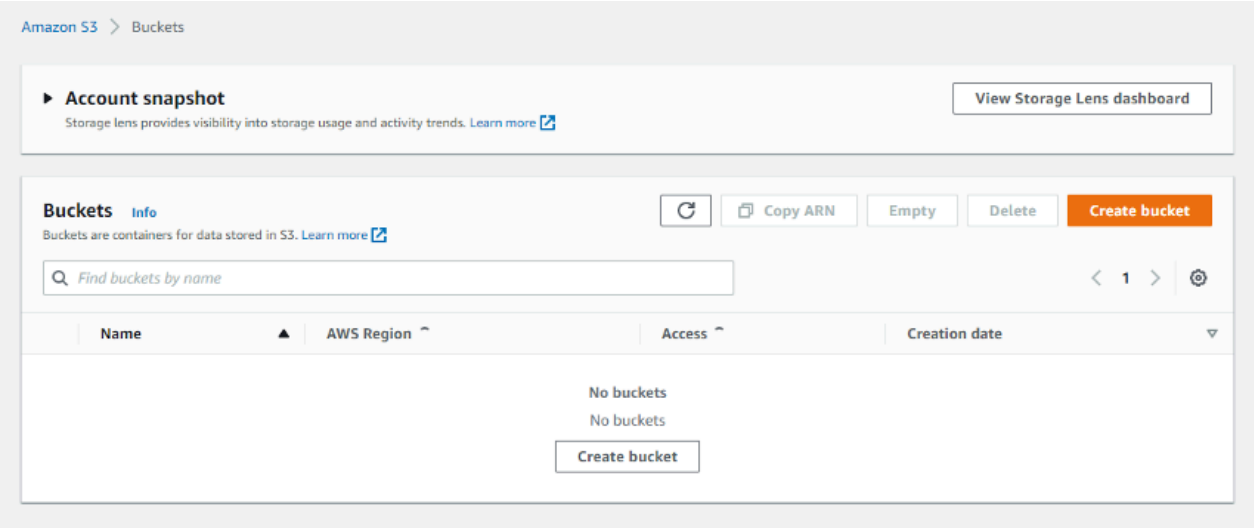
4.4 Delete a Bucket

The following task deletes a bucket that contains objects. The task deletes all objects, and then it deletes the bucket. The example works for buckets with or without versioning enabled.

Note: For buckets without versioning enabled, you can delete all objects directly and then delete the bucket. For buckets with versioning enabled, you must delete all object versions before deleting the bucket.

1. Download the file **DeleteBucket2.java** and copy-paste it under the Java package **com.amazonaws.samples** of your project – **S3Tasks**.
2. Replace "*** *Bucket name* ***" with the name of the bucket you created previously (e.g. sXXXXXXX-mybucket).



3. Right click on **DeleteBucket2.java** file and select **Run as>Java application**. The application will start to run. Wait until it finishes. When

you browse the AWS S3 Console, you will see that the bucket you previously created is now gone.